Cross-Language Expense Tracker

Prateek Sahay | 0.5 min

Prateek Sahay and Venkata Sai Makani

One application, two independent implementations: Python and C++17

Focus: equivalent behavior and meaningful language differences

18-minute plan: demonstrations, source examples, verification, and team Q&A

Shared features and observable behavior

Venkata Sai Makani | 0.83 min

Add expenses with a date, amount, category, description, and generated ID

List and filter by inclusive dates and an exact category

Search category or description without ASCII case sensitivity

Calculate category totals and an overall total; delete by ID

Persist records and reject invalid input without rewriting valid data

Architecture: separate interfaces, common contract

Prateek Sahay | 0.92 min

Command line -> parse options -> validate and load CSV -> core operation

Queries -> deterministic output; mutations -> temporary write -> replacement

Python: main.py interface; tracker.py core functions and dictionary records

C++: main.cpp interface; tracker.hpp declarations; tracker.cpp Tracker class

One behavioral suite exercises both programs and checks interoperability

CSV and exact currency are the shared contract

Venkata Sai Makani | 1.08 min

CSV fields: id, date, amount_cents, category, description

Integer cents: $0.10 + $0.20 = $0.30 without binary floating point

Valid dates: YYYY-MM-DD, years 0001-9999; real calendar validation

Positive amounts up to $999999999.99; up to 100,000 records

Printable ASCII text, quoted commas and doubled quotes, one record per line

Python live demonstration

Prateek Sahay | 2 min

Python | Recorded CLI demonstration

Selected actual captured program output

All expenses and invalid-date rejection

```
> list
ID | Date | Amount (USD) | Category | Description
1 | 2026-10-02 | $12.34 | Food | Lunch, campus
2 | 2026-10-02 | $5.00 | Transport | Bus ticket
3 | 2026-10-03 | $19.99 | Supplies | Notebook and
pens
4 | 2026-10-03 | $8.75 | Food | Coffee and breakfast
5 | 2026-10-04 | $15.00 | Transport | Trip to campus

> add --date 2026-02-30 ...
Error: Date must be a valid calendar date.
[exit 2]
```

Totals, combined filter, and search

```
> summary
Category | Total (USD)
Food | $21.09
Supplies | $19.99
Transport | $20.00
Overall | $61.08

> list --start 2026-10-03 --end 2026-10-04 --
category Food
ID | Date | Amount (USD) | Category | Description
4 | 2026-10-03 | $8.75 | Food | Coffee and breakfast

> search --query CAMPUS
ID | Date | Amount (USD) | Category | Description
1 | 2026-10-02 | $12.34 | Food | Lunch, campus
5 | 2026-10-04 | $15.00 | Transport | Trip to campus
```

Commands are abbreviated; executable/data paths and remaining invalid-add arguments are omitted. Full commands, all five add operations, and unchanged output are retained in companion .txt and evidence_results.json.

Live demo: three new expenses totaling $5.30

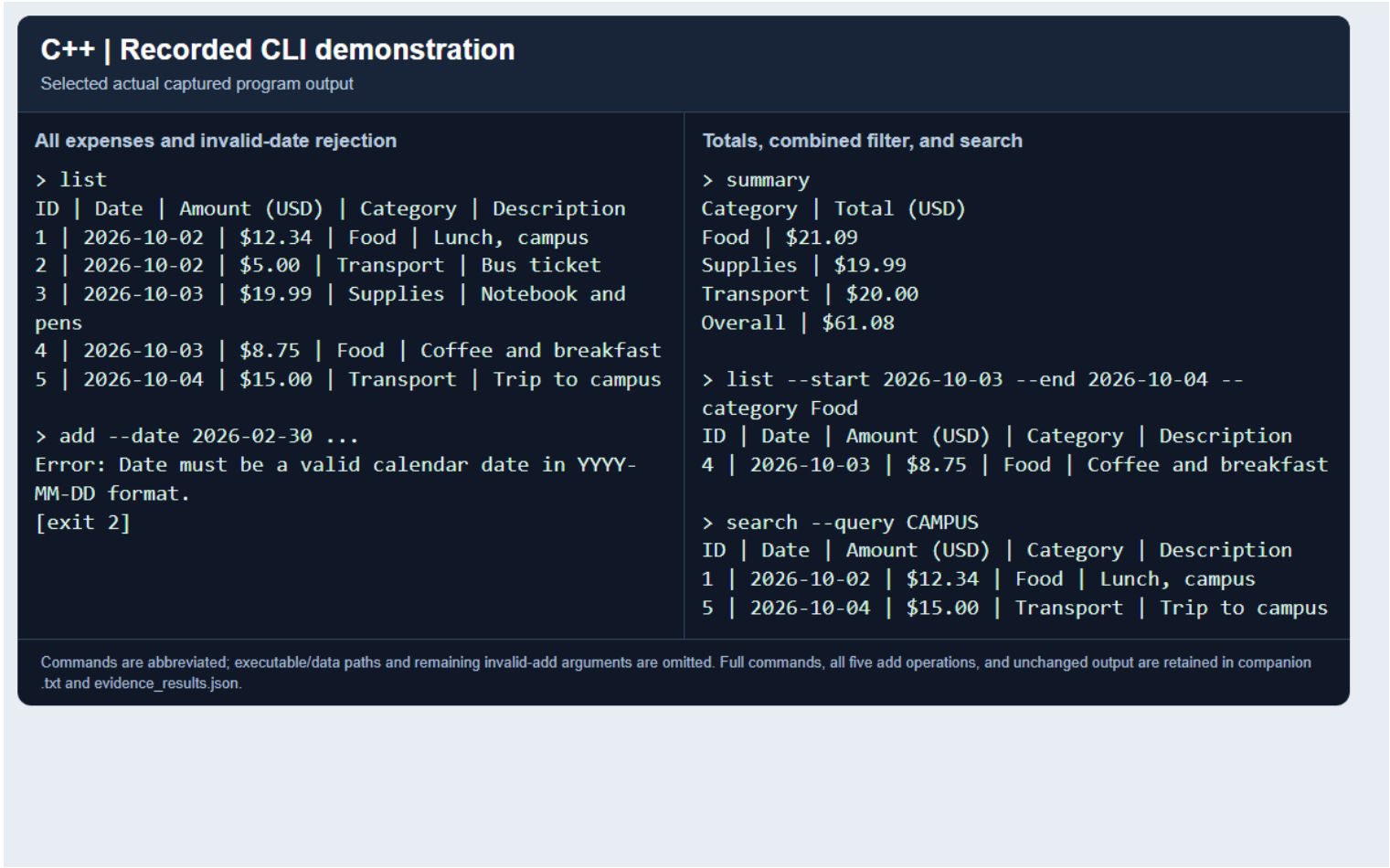
Show listing, inclusive filters, and search

Screenshot: five-record sample totaling $61.08

Reuse the live CSV for the C++ walkthrough

C++ live demonstration and interoperability

Venkata Sai Makani | 2 min



Read Python's live CSV with C++

Add a Books expense; expected total $25.29

Delete it, reject an invalid date, and verify $5.30

Screenshot: recorded five-expense sample ($61.08)

Types and containers shape the implementation

Prateek Sahay | 1.17 min

Python records are dictionaries; TypedDict documents their expected shape • Python type hints do not enforce field types at runtime • C++ Expense fields have compiler-enforced types and value ownership • Lists/dictionaries versus std::vector/std::map/std::set

python/tracker.py - Expense

```
class Expense(TypedDict):
    id: int
    date: date
    amount_cents: int
    category: str
    description: str
```

cpp/tracker.hpp - Expense

```
struct Expense {
    std::int32_t id;
    std::string date;
    std::int64_t amount_cents;
    std::string category;
    std::string description;
};
```

Filtering: comprehension versus explicit control flow

Venkata Sai Makani | 1.17 min

Python expresses selection with a list comprehension and predicate conditions • C++ uses a loop, std::optional bounds, and continue for rejected records • Both apply inclusive dates and the same exact category condition • Python returns references to dictionaries; C++ copies selected Expense values

python/tracker.py - filter_expenses excerpt

```
    return [
        expense for expense in expenses
        if (start_date is None or expense["date"] >= start_date)
        and (end_date is None or expense["date"] <= end_date)
        and (selected_category is None or expense["category"] == selected_category)
        and (selected_query is None or selected_query in expense["category"].lower()
             or selected_query in expense["description"].lower())
    ]
```

cpp/tracker.cpp - Tracker::select excerpt

```
    for (const auto& record : records_) {
        if ((filters.start && record.date < *filters.start) ||
            (filters.end && record.date > *filters.end) ||
            (normalized_category && record.category != *normalized_category)) {
            continue;
        }
```

Aggregation and checked integer totals

Prateek Sahay | 1.08 min

Python dict.get supplies a default when a category is first encountered • C++ std::map indexes a category and updates its integer total • Python sorts categories when printing; C++ map iteration is already sorted • Both check the shared signed 64-bit total bound and format integer cents

python/tracker.py - summarize

```
def summarize(expenses: list[Expense]) -> tuple[dict[str, int], int]:
    totals: dict[str, int] = {}
    overall = checked_total(expenses)
    for expense in expenses:
        category = expense["category"]
        totals[category] = totals.get(category, 0) + expense["amount_cents"]
    return totals, overall
```

cpp/tracker.cpp - Tracker::summarize excerpt

```
    Summary summary;
    for (const auto& record : select(filters)) {
        auto& category_total = summary.by_category[record.category];
        category_total = checked_add(category_total, record.amount_cents);
        summary.overall = checked_add(summary.overall, record.amount_cents);
    }
    return summary;
```

Resource ownership: context managers and RAII

Venkata Sai Makani | 1.17 min

Python with closes files; try/finally removes temporary files • C++ streams and a temporary-file destructor release resources on scope exit • std::vector and std::string own allocations without manual new/delete • Successful writes replace the target; atomic replacement is not writer locking

python/tracker.py - load_expenses excerpt

```
        with path.open("r", encoding="utf-8", newline="") as source:
            reader = csv.reader(checked_csv_lines(source), strict=True)
```

cpp/tracker.cpp - TemporaryFile destructor

```
    ~TemporaryFile() {
        if (active_) {
            std::error_code ignored;
            std::filesystem::remove(path_, ignored);
        }
    }
```

Error handling protects the user and stored data

Prateek Sahay | 1.08 min

Python raises TrackerError; C++ throws runtime_error at validation boundaries • Both interfaces print Error: ... to stderr and return exit code 2 • Reject invalid dates, signs/exponents, duplicate flags/IDs, and malformed CSV • Complete validation happens before saving a changed file

python/main.py - main error boundary

```
    except TrackerError as exc:
        print(f"Error: {exc}", file=sys.stderr)
        return 2
```

cpp/main.cpp - main error boundary

```
    } catch (const std::exception& exception) {
        std::cerr << "Error: " << exception.what() << '\n';
        return 2;
    }
```

Modularity and development workflow

Venkata Sai Makani | 0.83 min

Python imports reusable core functions from a separate module • C++ separates the public interface in a header from its implementation • Python runs with an interpreter; C++ builds an executable with a C++17 compiler • Git branches, peer review, and CI support consistent requirements

python/main.py - imported core interface

```
from tracker import (
    TrackerError, add_expense, delete_expense, filter_expenses,
    format_amount, load_expenses, save_expenses, summarize,
)
```

cpp/tracker.hpp - Tracker public API excerpt

```
class Tracker {
public:
    static Tracker load(const std::filesystem::path& file);

    std::int32_t add(const std::string& date, const std::string& amount,
                     const std::string& category, const std::string& description);
```

Paradigms appear in the actual code

Prateek Sahay | 1.08 min

Python combines procedural functions with declarative comprehensions • C++ combines encapsulated object state with generic STL algorithms • Both languages support several paradigms; these are project choices • Deletion shows explicit iteration versus a predicate-driven algorithm

python/tracker.py - delete_expense excerpt

```
    for index, expense in enumerate(expenses):
        if expense["id"] == expense_id:
            del expenses[index]
            return expense_id
    raise TrackerError(f"Expense #{expense_id} does not exist.")
```

cpp/tracker.cpp - Tracker::erase excerpt

```
    const auto record = std::find_if(records_.begin(), records_.end(),
                                     [id](const Expense& expense) { return expense.id == id; });
    if (record == records_.end()) {
        throw std::runtime_error("Expense ID was not found.");
    }
    records_.erase(record);
```

Performance: expectations and measurement limits

Venkata Sai Makani | 0.92 min

Measured here: correctness tests pass and the C++ build has no warnings

No controlled benchmark establishes a runtime or memory winner

Both read the entire CSV; mutations rewrite it and filtering scans records

Expected tradeoffs: interpreter/dictionary overhead versus native values and copies

Python groups with a dict then sorts; C++ updates an ordered map

Verification: shared behavior and failure cases

Prateek Sahay / Venkata Sai Makani | 0.92 min

55 integration tests: 25 Python, 25 C++, and 5 interoperability tests

7 additional Python core/storage tests

Coverage: exact cents, leap years, limits, filters, search, deletion, and CSV quoting

Failure checks preserve the original bytes; Python simulates replacement failure

C++ must be built first so parity tests run instead of being skipped

Limitations and lessons from the comparison

Prateek Sahay / Venkata Sai Makani | 0.75 min

Scope: single writer, USD, printable ASCII, bounded local CSV storage

No concurrent writes, GUI, database, authentication, or benchmark results

Shared validation and tests prevent subtle cross-language divergence

Language choice should follow constraints: iteration speed, ownership, and deployment

Conclusion and team Q&A

Prateek Sahay / Venkata Sai Makani | 0.5 min

Same core behavior, interoperable data, different language design choices

Actual code connects types, containers, paradigms, errors, and resource ownership

Source, README, reports, screenshots, and test instructions are in the repository

Repository: https://github.com/psahay101/cross-language-expense-tracker

Questions: both presenters can navigate the source or repeat a demo command